UNIT–2: AGENT STATE REPRESENTATION (DETAILED)

🔶 1. AGENT STATE REPRESENTATION

🔹 Definition

Agent State Representation is the method of encoding the **current situation of the environment** so that the agent can **select optimal actions**.

🔹 Core Idea

- A **good state** contains all relevant information
- Must satisfy **Markov Property** (no need of past history)

🔹 Key Elements

State (S): The current situation of the environment that contains all information needed for decision-making.

Action (A): The set of possible moves or decisions the agent can take in a given state.

Reward (R): The immediate feedback (numerical value) received after taking an action.

Policy (π): A strategy that defines how an agent chooses actions in each state.

Environment: The external system with which the agent interacts and from which it receives states and rewards.

🔹 Working (Conceptual Flow)

1. Agent observes environment → gets state
2. Chooses action based on policy
3. Receives reward + next state
4. Updates knowledge

🔹 Example

Self-driving car:

- State = position, speed, nearby objects
- Action = accelerate, brake

🔹 Applications / Real-Life Uses

- Robotics navigation
- Game AI (Chess, Go)
- Autonomous vehicles
- Recommendation systems
- Industrial automation

🔹 Advantages

1. Enables structured decision making
2. Simplifies complex environments
3. Supports RL algorithms

4. Improves learning efficiency
5. Helps generalization

🔹 **Disadvantages**

1. Difficult to design optimal state
2. High-dimensional states are complex
3. May lose information (if simplified)
4. Requires domain knowledge
5. Poor representation → poor performance

🔶 **2. MARKOV DECISION PROCESS (MDP)**

🔹 **Definition**

MDP is a mathematical framework for modeling **sequential decision-making problems** under uncertainty.

🔹 **Core Idea**

👉 Future depends only on **current state + action**

🔹 **Components (Must Write)**

- S → States
- A → Actions
- P → Transition probability
- R → Reward
- γ → Discount factor

👉 MDP = (S, A, P, R, γ)

🔹 **Working**

1. Agent in state S
2. Takes action A
3. Moves to next state S'
4. Receives reward R
5. Process repeats

🔹 **Example**

Robot path planning:

- States = positions
- Actions = move directions

🔹 **Applications**

- Robotics
- Game playing (AlphaGo)

- Finance (trading strategies)
- Inventory management
- Healthcare decision systems

🔹 **Advantages**

1. Strong mathematical foundation
2. Handles uncertainty well
3. Supports optimal decision making
4. Basis of RL algorithms
5. Flexible modeling

🔹 **Disadvantages**

1. Requires full environment model
2. Large state space problem
3. Computationally expensive
4. Difficult for real-world complexity
5. Assumes Markov property (may not hold)

🔶 **3. MARKOV PROCESS (MP)**

🔹 **Definition**

A Markov Process is a stochastic process where **next state depends only on current state**.

🔹 **Core Idea**

👉 "Memoryless system"

🔹 **Working**

- State transitions based on probability
- No actions, no rewards

🔹 **Example**

Weather system:

- Today → Tomorrow

🔹 **Applications**

- Weather prediction
- Stock market modeling
- Queue systems
- Reliability analysis
- Biology (gene sequences)

🔹 **Advantages**

1. Simple model
2. Easy to analyze
3. Mathematical clarity
4. Useful for prediction
5. Basis for advanced models

🔹 **Disadvantages**

1. No decision-making capability
2. No reward mechanism
3. Limited real-world use
4. Assumes Markov property
5. Cannot model complex dependencies

🔶 **4. MARKOV REWARD PROCESS (MRP)**

🔹 **Definition**

MRP is a Markov Process extended with **reward signals**.

🔹 **Core Idea**

👉 "States + rewards → evaluate value"

🔹 **Components**

- S, P, R, γ

🔹 **Value Function**

$$V(s) = \mathbb{E}[R_t + \gamma V(s_{t+1})]$$

🔹 **Working**

- Agent transitions between states
- Collects rewards
- Computes expected value

🔹 **Example**

Game scoring system

🔹 **Applications**

- Reward evaluation systems
- Game analysis
- Economic modeling
- Risk analysis
- Performance evaluation

🔹 **Advantages**

1. Includes reward structure
2. Helps evaluate states
3. Useful in planning
4. Foundation of RL
5. Simple extension of MP

🔹 **Disadvantages**

1. No actions included
2. Cannot control decisions
3. Requires known probabilities
4. Limited flexibility
5. Not suitable for full RL tasks

🔶 **5. MARKOV CHAIN**

🔹 **Definition**

Markov Chain is a sequence of states where **transition depends only on current state**.

🔹 **Core Idea**

👉 "Chain of probabilistic states"

🔷 **Properties of a Markov Chain (Short)**

1. **Markov Property:** Future depends only on present state (memoryless).
2. **Transition Probability:** Movement between states is defined by probabilities.
3. **State Space:** Set of all possible states.
4. **Transition Matrix:** Matrix representing transition probabilities.
5. **Time Homogeneity:** Probabilities do not change over time.
6. **Absorbing State:** Once entered, cannot be left.
7. **Irreducibility:** Every state is reachable from any other state.
8. **Ergodicity:** System reaches stable long-term behavior.

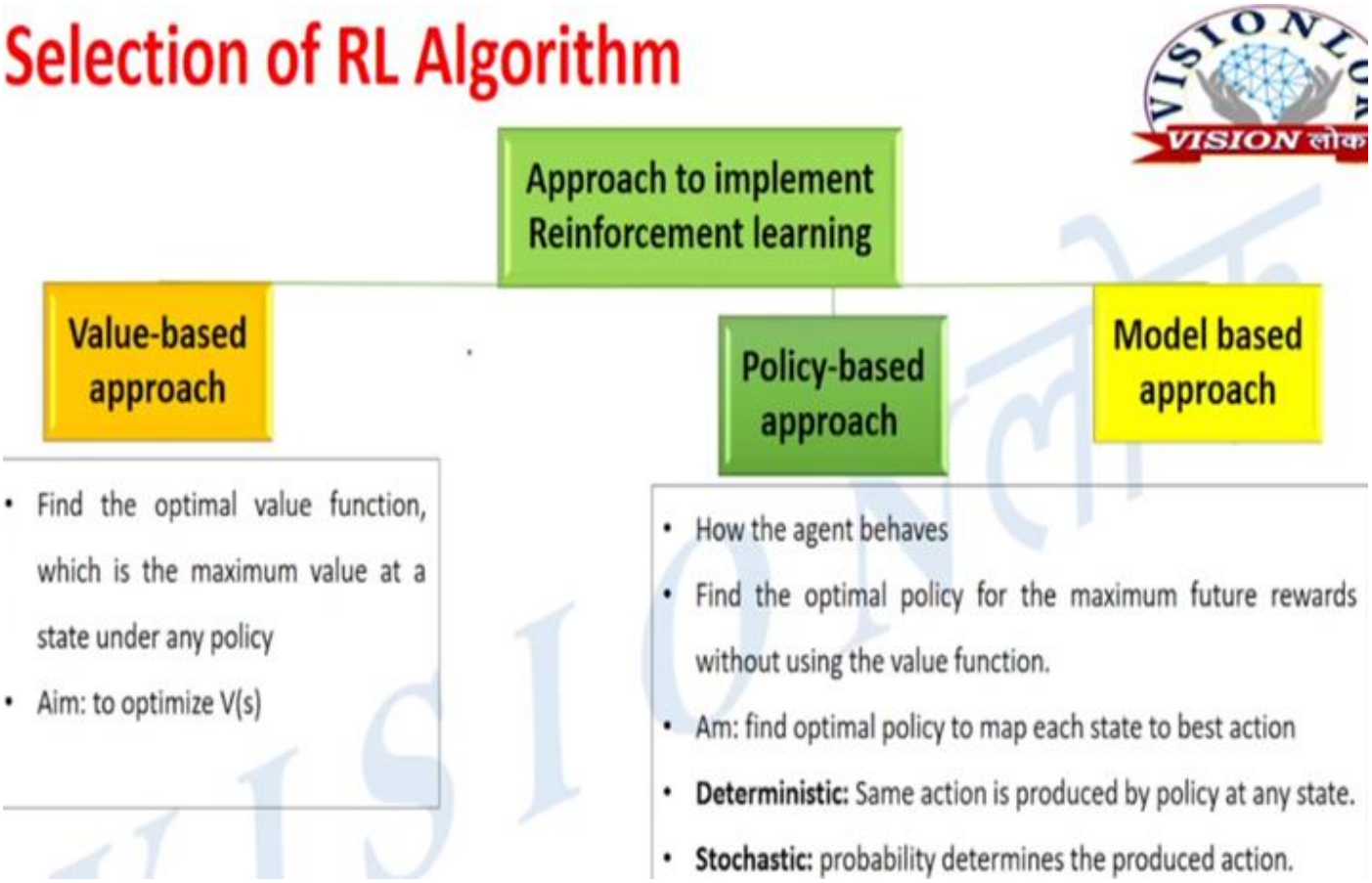
🔹 **Working**

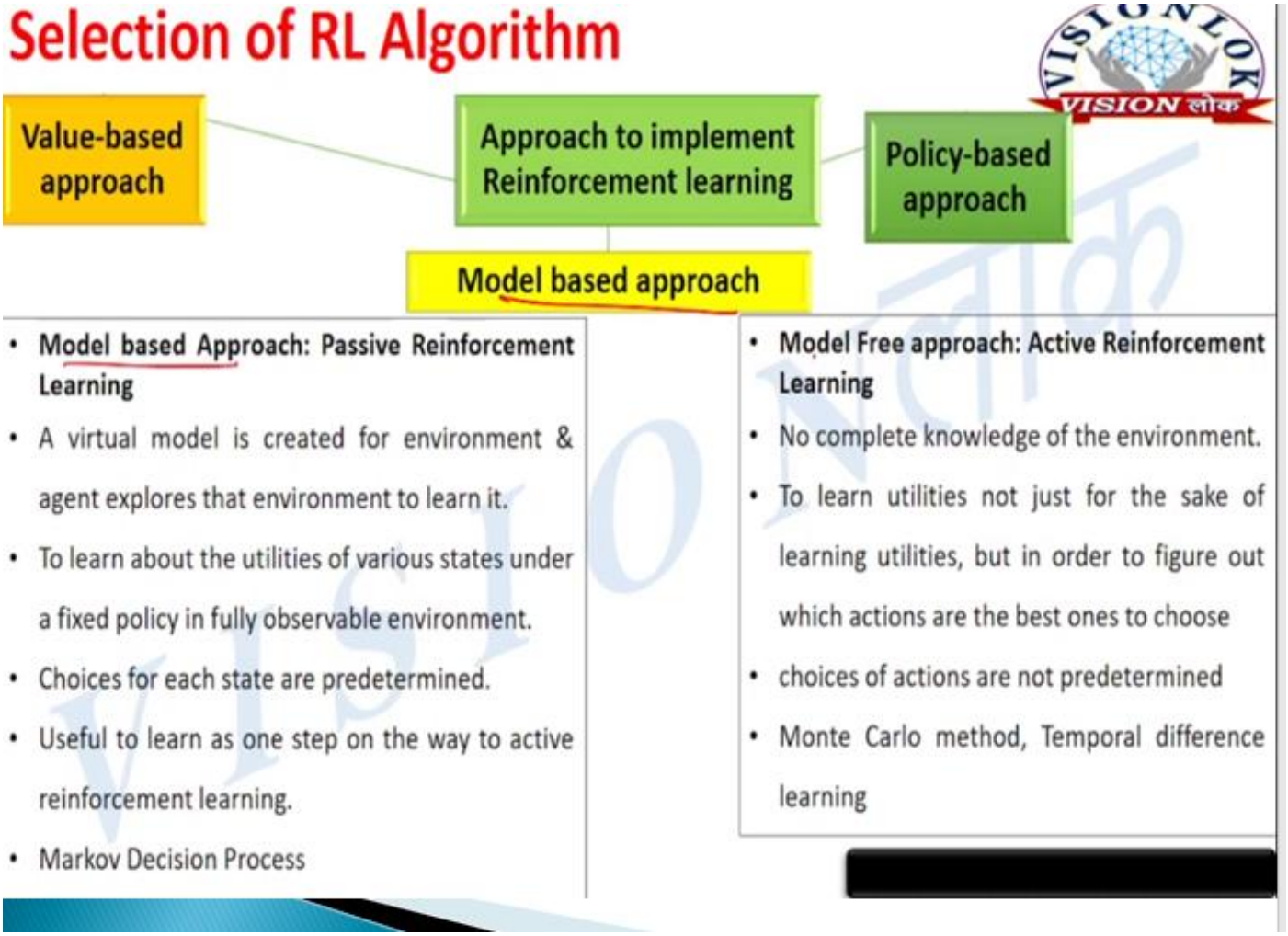
- States connected via probabilities
- Represented using transition matrix

🔹 **Example**

Weather transitions

- **Applications**
  - Text generation
  - Speech recognition
  - Web ranking (PageRank)
  - Finance modeling
  - Biology
- **Advantages**
  1. Easy to implement
  2. Strong mathematical base
  3. Useful in prediction
  4. Widely applicable
  5. Efficient for simple systems
- **Disadvantages**
  1. No reward or action
  2. Limited decision-making
  3. Assumes independence
  4. Not suitable for complex tasks
  5. Oversimplification





🔶 6. BELLMAN EQUATIONS (VERY IMPORTANT)

🔹 Definition

Bellman Equation expresses the value of a state as **immediate reward + discounted future value**.

🔹 Core Idea

👉 "Recursive relationship"

🔹 Bellman Equation

$$V(s) = \max_a \sum P(s' \mid s, a)[R + \gamma V(s')]$$

🔹 Working

- Breaks problem into subproblems
- Computes optimal policy

🔹 Example

Shortest path problem

🔹 Applications

- Dynamic programming
- RL algorithms (Q-learning)
- Game AI

- Robotics
- Optimization problems

🔹 **Advantages**

1. Core of RL theory
2. Enables optimal decisions
3. Recursive structure
4. Supports dynamic programming
5. Efficient solution method

🔹 **Disadvantages**

1. Requires full model knowledge
2. Computationally expensive
3. Large state space issue
4. Hard for real-world problems
5. Approximation needed

🔶 **7. PARTIALLY OBSERVABLE MDP (POMDP)**

🔹 **Definition**

POMDP is an extension of MDP where **agent cannot fully observe the environment state**.

🔹 **Core Idea**

👉 "Uncertainty in observation"

🔹 **Components**

- S, A, P, R, γ + O (observations)

🔹 **Working**

1. Agent receives partial observation
2. Maintains belief state
3. Chooses action

🔹 **Example**

Poker game

🔹 **Applications**

- Robotics (sensor uncertainty)
- Healthcare diagnosis
- Autonomous driving
- Finance
- Surveillance systems

🔹 **Advantages**

1. Real-world applicability
2. Handles uncertainty
3. Flexible modeling
4. Better decision making
5. Robust system

🔹 **Disadvantages**

1. Very complex
2. High computation cost
3. Difficult to implement
4. Needs approximation
5. Hard to scale

🔶 **8. PLANNING BY DYNAMIC PROGRAMMING (DP)**

🔹 **Definition (Exam Ready)**

Dynamic Programming in RL is a set of methods used to **solve MDPs** by **breaking problems into smaller subproblems** using the **Bellman equations**.

🔹 **Core Idea**

👉 "Solve complex problem step-by-step using recursion"
👉 Requires **complete model (P, R known)**

🔹 **Key Concepts**

- Bellman equations
- Value function
- Policy
- Iterative updates

🔹 **Working (General DP Flow)**

1. Initialize value function
2. Apply Bellman updates
3. Improve policy
4. Repeat until convergence

🔹 **Applications**

- Robotics planning
- Game AI (Chess, Gridworld)
- Resource allocation
- Route optimization

🔹 **Advantages**

1. Guarantees optimal solution (if model known)
2. Strong mathematical foundation
3. Systematic approach
4. Works well for small problems
5. Basis of many RL algorithms

🔹 **Disadvantages**

1. Requires full environment model
2. Computationally expensive
3. Not scalable to large state spaces
4. Memory intensive
5. Not suitable for unknown environments

🔶 9. POLICY EVALUATION

🔹 **Definition**

Policy Evaluation computes the **value function V(s)** for a given policy π.

🔹 **Core Idea**

👉 "How good is this policy?"

🔹 **Bellman Expectation Equation**

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a)[R + \gamma V^{\pi}(s')]$$

🔹 **Working / Steps**

1. Initialize V(s) arbitrarily
2. Update values using Bellman equation
3. Repeat until values converge

🔹 **Example**

Evaluating a fixed strategy in a game

🔹 **Applications**

- Policy analysis
- Game strategy evaluation
- Robotics decision validation
- Financial planning
- Control systems

🔹 **Advantages**

1. Evaluates policy performance
2. Simple iterative method
3. Works for any policy
4. Helps improve decisions
5. Foundation of policy iteration

🔹 **Disadvantages**

1. Requires known model
2. Slow convergence
3. Computational cost
4. Not scalable
5. Needs repeated updates

🔶 **10. VALUE ITERATION**

🔹 **Definition**

Value Iteration is a DP method that **directly computes optimal value function** by repeatedly applying Bellman optimality equation.

🔹 **Core Idea**

👉 "Find best value → then derive policy"

🔹 **Bellman Optimality Equation**

$$V(s) = \max_a \sum_{s'} P(s' \mid s, a)[R + \gamma V(s')]$$

🔹 **Working / Steps**

1. Initialize V(s) = 0
2. Update values using max over actions
3. Repeat until convergence
4. Extract optimal policy

🔹 **Example**

Finding shortest path in grid

🔹 **Applications**

- Navigation systems
- Robotics
- Game AI
- Path planning
- Resource optimization

🔹 **Advantages**

1. Finds optimal policy directly
2. Simple to implement
3. Converges faster than policy iteration
4. Efficient for small problems
5. No separate evaluation step

🔹 **Disadvantages**

1. Requires full model
2. Computationally expensive
3. Large state space issue
4. Slow for complex problems
5. Memory intensive

🔹 **Exam Notes**

- "Value iteration = optimize values first"

🔶 **11. POLICY ITERATION**

🔹 **Definition**

Policy Iteration is a DP method that **alternates between policy evaluation and policy improvement**.

🔹 **Core Idea**

👉 "Evaluate → Improve → Repeat"

🔹 **Steps (VERY IMPORTANT)**

Step 1: Policy Evaluation

- Compute V(s) for current policy

Step 2: Policy Improvement

- Update policy using:

$$\pi'(s) = \arg\max_a \sum P(s' \mid s, a)[R + \gamma V(s')]$$

Step 3:

- Repeat until policy stops changing

🔹 **Working Summary**

- Evaluate current policy
- Improve it
- Repeat → optimal policy

🔹 **Example**

Improving game strategy step-by-step

- **Applications**
  - Game AI
  - Robotics control
  - Decision-making systems
  - Optimization problems
  - Resource planning
- **Advantages**
  1. Guaranteed optimal policy
  2. Faster convergence in many cases
  3. Clear two-step process
  4. Stable learning
  5. Efficient for medium problems
- **Disadvantages**
  1. Requires full model
  2. Policy evaluation is expensive
  3. Not scalable
  4. Memory heavy
  5. Complex implementation
- **Exam Notes**
  - “Policy Iteration = Evaluation + Improvement”

🔷 Policy Evaluation vs Value Iteration vs Policy Iteration

Point	Policy Evaluation	Value Iteration	Policy Iteration
1. Purpose	Evaluates a given policy	Finds optimal value function	Finds optimal policy
2. Policy Update	No policy update	Policy derived at end	Policy updated every iteration
3. Equation Used	Bellman Expectation	Bellman Optimality	Both (Evaluation + Optimality)
4. Process Type	Single step (evaluation only)	Single loop (value update)	Two steps (evaluation + improvement)
5. Convergence	Slow (no optimization)	Faster per iteration	Fewer iterations but heavier computation

🔷 UNIT–3: MODEL-FREE LEARNING (REINFORCEMENT LEARNING)

🔶 1. MODEL-FREE LEARNING (Overview)

🔹 Definition (Exam Ready)

Model-free learning refers to RL methods where the agent **learns directly from interaction with the environment** without knowing transition probabilities (P) or reward function (R).

🔹 Core Idea

👉 "Learn by experience, not by model"

🔹 Key Concepts

- No environment model
- Trial-and-error learning
- Value estimation
- Policy learning

🔹 Example

Learning to play a game by trying moves

🔹 Applications

- Robotics (real-time learning)
- Game AI (Atari, Chess)
- Recommendation systems
- Autonomous driving
- Stock trading

🔹 Advantages

1. No need of environment model
2. Works in unknown environments
3. Flexible learning
4. Suitable for real-world problems
5. Simpler implementation

🔹 Disadvantages

1. Requires large data
2. Slow convergence
3. High variance
4. Less stable
5. Exploration needed

🔹 Exam Notes

- "Model-free = no P, no R known"

🔶 2. MONTE CARLO (MC) LEARNING

🔹 Definition

Monte Carlo learning estimates value functions using **complete episodes (end-to-end experience)**.

🔹 Core Idea

👉 "Learn after episode ends"

🔹 Key Concepts

- Episode-based learning
- Returns (G)
- No bootstrapping

🔹 Formula (Return)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

🔹 Working / Steps

1. Run full episode
2. Observe rewards
3. Calculate return
4. Update value

🔹 Example

Game ends → calculate total reward

🔹 Applications

- Game playing
- Simulation systems
- Risk analysis
- Finance
- Episodic tasks

🔹 Advantages

1. Simple concept
2. No model required
3. Works with real experience
4. Unbiased estimates
5. Easy to implement

🔹 Disadvantages

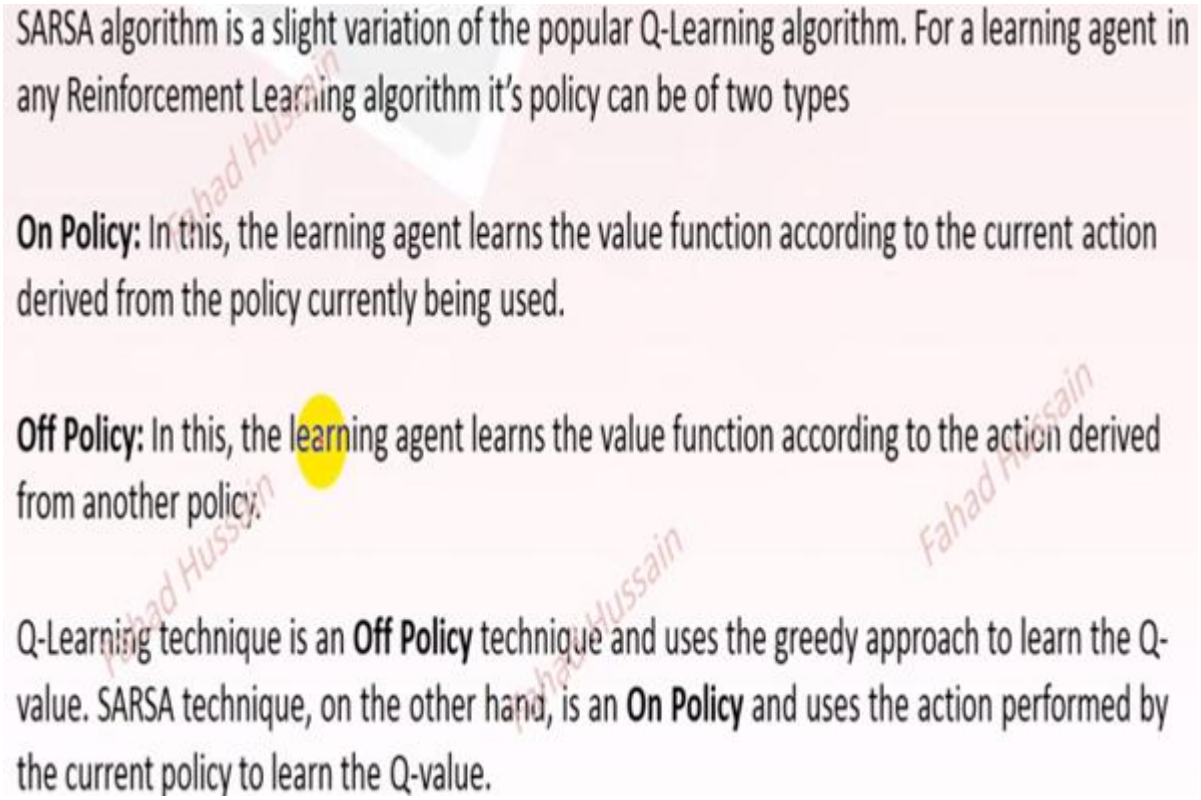
1. Must wait till episode ends
2. High variance

3. Slow learning
4. Not suitable for continuous tasks
5. Memory intensive

🔹 **Exam Notes**

- "MC = learn from complete episodes"

🔶 **3. SARSA (State–Action–Reward–State–Action)**



🔹 **Definition**

SARSA is an **on-policy RL algorithm** that updates Q-values using the **current policy's action**.

🔹 **Core Idea**

👉 "Learn from actions actually taken"

🔹 **Formula**

$$Q(s,a) = Q(s,a) + \eta[R + \gamma Q(s',a') - Q(s,a)]$$

🔹 **Working / Steps**

1. Take action A in state S
2. Observe reward R and next state S'
3. Choose next action A'
4. Update Q-value
5. Repeat

🔹 **Example**

Learning safe driving policy

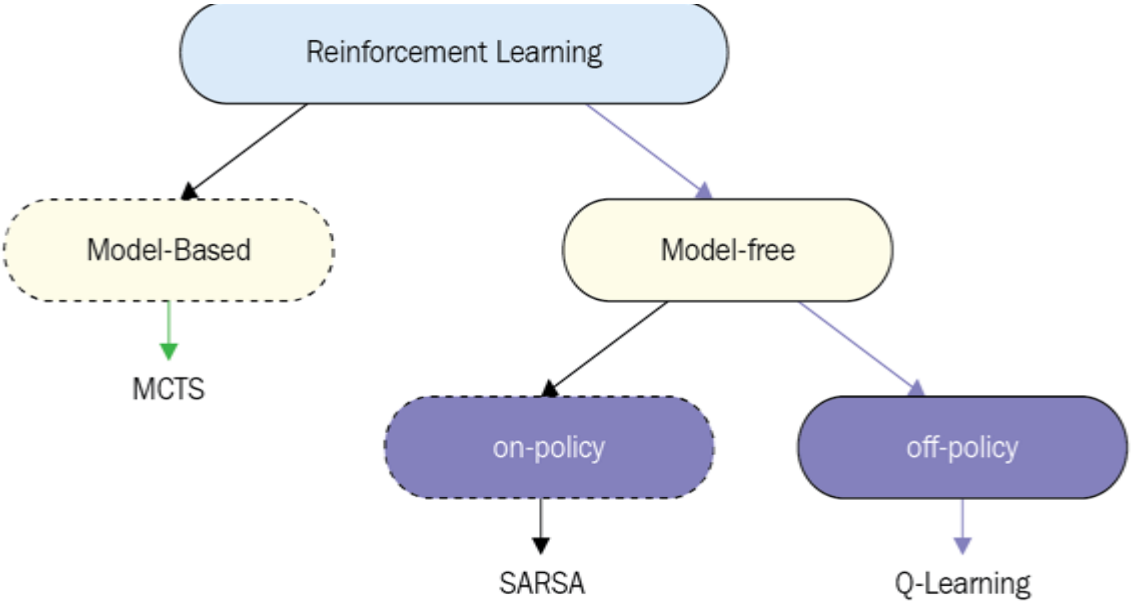
- **Applications**
  - Robotics
  - Game AI
  - Control systems
  - Navigation
  - Industrial automation
- **Advantages**
  1. Stable learning
  2. Considers current policy
  3. Less risky behavior
  4. Works in real-time
  5. Handles exploration well
- **Disadvantages**
  1. Slower convergence
  2. Depends on policy
  3. May not find optimal solution
  4. Sensitive to parameters
  5. Needs careful tuning
- **Exam Notes**
  - SARSA = **on-policy**

4. Q-LEARNING

•**Q-learning** is a **model-free** reinforcement learning algorithm.

•Q-learning is a **values-based** learning algorithm. Value based algorithms updates the value function based on an equation(particularly Bellman equation). Whereas the other type, **policy-based** estimates the value function with a greedy policy obtained from the last policy improvement.

•Q-learning is an **off-policy learner**. Means it learns the value of the optimal policy independently of the agent's actions. On the other hand, an **on-policy learner** learns the value of the policy being carried out by the agent, including the exploration steps and it will find a policy that is optimal, taking into account the exploration inherent in the policy.



Exploration is when the Agent tries to improve its knowledge about each action it can take that would lead to long-term benefits.

Exploitation is when the Agent exploits the current knowledge by being greedy to choose the greedy approach to get the most reward.

Q-learning is a model-free, value-based, off-policy learning algorithm.

- **Model-free**: The algorithm that estimates its optimal policy without the need for any transition or reward functions from the environment.
- **Value-based:** Q learning updates its value functions based on equations, (say Bellman equation) rather than estimating the value function with a greedy policy.
- **Off-policy:** The function learns from its own actions and doesn't depend on the current policy.

🔹 **Definition**

Q-Learning is an **off-policy RL algorithm** that learns optimal policy **independent of agent's actions**.

🔹 **Core Idea**

👉 "Learn best action, not actual action"

🔹 **Formula**

$$Q(s,a) = Q(s,a) + \eta[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

🔹 **Working / Steps**

1. Observe state S
2. Choose action A
3. Receive reward R, next state S'
4. Update using max Q
5. Repeat

🔹 **Example**

Finding shortest path

🔹 **Applications**

- Robotics
- Game AI
- Route optimization
- Finance
- Recommendation systems

🔹 **Advantages**

1. Finds optimal policy
2. Faster convergence
3. Independent of behavior policy
4. Widely used
5. Simple implementation

🔹 **Disadvantages**

1. Overestimation problem
2. Requires exploration
3. High variance
4. Needs tuning
5. Not stable in some cases

🔹 **Exam Notes**

- Q-learning = **off-policy**

🔶 **5. INCREMENTAL UPDATE**

🔹 **Definition**

Incremental Update updates value estimates **step-by-step without storing all past data**.

🔹 **Core Idea**

👉 "Update gradually using new data"

🔹 **Formula**

$$V(s) = V(s) + \eta(Target - V(s))$$

🔹 **Working**

- Update after each step
- No need to store full history

🔹 **Example**

Updating average marks after each test

🔹 **Applications**

- Online learning
- Real-time systems
- Streaming data
- Robotics
- Finance

🔹 **Advantages**

1. Memory efficient
2. Fast updates
3. Real-time learning
4. Scalable
5. Simple

🔹 **Disadvantages**

1. Sensitive to learning rate
2. May be unstable
3. Requires tuning
4. Slower convergence
5. Noise sensitive

🔹 **Exam Notes**

- “Incremental = step-by-step update”

🔶 **6. EXPLORATION vs EXPLOITATION**

🔹 **Definition**

Exploration means trying new actions, while exploitation means choosing **best-known action**.

🔹 **Core Idea**

👉 Balance between:

- Exploring new actions
- Using best known action

🔹 **Techniques**

- ε-greedy method
- Softmax selection

🔹 **Example**

Trying new restaurant vs going to favorite

🔹 **Applications**

- Recommendation systems

- Robotics
- Game AI
- Marketing strategies
- Online learning

🔷 SARSA vs Q-Learning (5 Differences)

Feature	SARSA	Q-Learning
1. Type	On-policy algorithm	Off-policy algorithm
2. Learning Basis	Uses **actual action taken (a')**	Uses **best possible action (max Q)**
3. Update Rule	$Q(s,a) \leftarrow R + \gamma Q(s',a')$	$Q(s,a) \leftarrow R + \gamma \max_{a'} Q(s',a')$
4. Behavior	More **conservative / safe**	More **aggressive / optimal**
5. Convergence	Slower	Faster

🔶 7. TEMPORAL DIFFERENCE (TD) LEARNING

> **Temporal Difference Learning (TD Learning)** is a method in **reinforcement learning** where an agent learns by **updating its predictions based on the difference between expected and actual outcomes over time**.
>
> It learns **step-by-step from experience**, without waiting for the final result.
>
> **Simple Idea**
> TD Learning updates its knowledge using:
>
> $$New\ Value = Old\ Value + \alpha(Reward + \gamma \times Next\ Value - Old\ Value)$$
>
> Where:
> •**α (alpha)** = learning rate
> •**γ (gamma)** = discount factor (importance of future reward)
> •**Reward** = immediate feedback
> •**Next Value** = estimated value of next state
> The key part is the **Temporal Difference (TD Error):**
>
> $$TD\ Error = Reward + \gamma \times Next\ Value - Old\ Value$$
>
> It measures **how wrong the prediction was**.

🔹 Definition (Exam Ready)

Temporal Difference (TD) Learning is a model-free RL method that **learns from incomplete episodes** by combining ideas of **Monte Carlo (MC)** and **Dynamic Programming (DP)**.

🔹 Core Idea

👉 "Learn step-by-step using current estimate + next estimate"
👉 Called **bootstrapping**

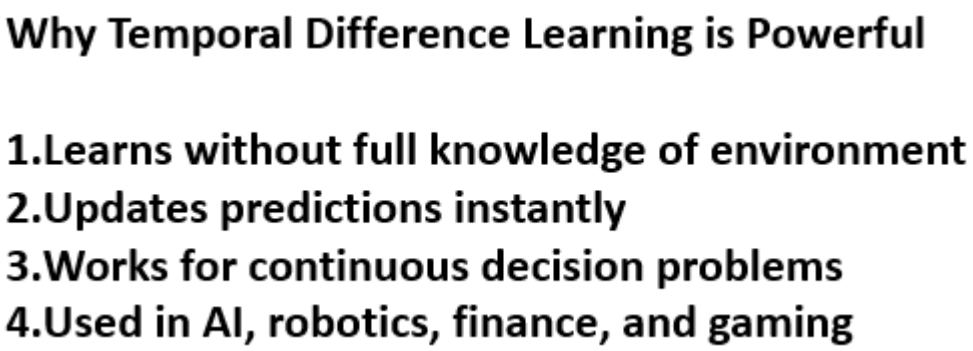
- **Key Concepts**
  - Bootstrapping
  - Online learning
  - TD Error
  - Value function update
- **TD Error (VERY IMPORTANT)**

$$\delta = R + \gamma V(s') - V(s)$$

- **Update Rule**

$$V(s) = V(s) + \eta\delta$$

- **Working / Steps**
  1. Observe current state S
  2. Take action → get reward R and next state S'
  3. Compute TD error
  4. Update value function
  5. Repeat



- **Example**

Learning driving step-by-step without waiting for journey end

- **Applications**
  - Robotics
  - Online recommendation systems
  - Game AI
  - Finance prediction
  - Real-time control systems
- **Advantages**
  1. Learns without waiting for episode end
  2. Faster than Monte Carlo
  3. Low memory usage

4. Works in continuous tasks
5. Efficient learning

🔹 **Disadvantages**

1. Biased estimates
2. Sensitive to parameters
3. May be unstable
4. Requires tuning
5. Convergence issues

🔶 **8. TD-λ (TD-LAMBDA)**

🔹 **Definition**

TD-λ is an extension of TD learning that combines **multiple-step returns** using a parameter λ (lambda).

🔹 **Core Idea**

👉 "Mix of short-term and long-term learning"
👉 λ controls balance

🔹 **Key Concepts**

- λ (0 ≤ λ ≤ 1)
- Eligibility traces
- Multi-step learning

🔹 **Special Cases (VERY IMPORTANT)**

- λ = 0 → TD(0)
- λ = 1 → Monte Carlo

🔹 **Formula (Concept)**

$$V(s) = V(s) + \eta \sum_t (\gamma\lambda)^t \delta_t$$

🔷 TD Learning vs Monte Carlo Method (5 Differences)

Feature	Temporal Difference (TD) Learning	Monte Carlo (MC) Method
1. Learning Type	Learns **step-by-step (online)**	Learns **after complete episode**
2. Update Timing	Updates after every step	Updates only at end of episode
3. Bootstrapping	Uses **bootstrapping** (next state value)	**No bootstrapping**
4. Speed	Faster learning	Slower learning
5. Variance/Bias	Higher bias, lower variance	Low bias, high variance

🔶 9. ELIGIBILITY TRACES

🔹 Definition

Eligibility Traces are a mechanism to **assign credit to past states/actions** for current rewards.

🔹 Core Idea

👉 "Recent states get more credit"

🔹 Key Concepts

- Decaying memory
- Credit assignment
- Trace decay

🔹 Formula

$$E(s) = \gamma\lambda E(s) + 1$$

🔹 Working / Steps

1. Initialize traces
2. Update trace for visited states
3. Decay old traces
4. Update values using traces

🔹 Example

Reward for goal → credit given to previous steps

🔹 Applications

- Robotics navigation
- Game learning
- Sequence prediction
- Speech recognition
- Control systems

🔹 Advantages

1. Faster learning
2. Better credit assignment
3. Improves TD learning
4. Handles delayed rewards
5. Efficient updates

🔹 Disadvantages

1. Extra memory required
2. More computation

3. Parameter tuning needed
4. Complex implementation
5. Can be unstable

🔶 10. EPSILON-GREEDY POLICY

🔹 Definition (Exam Ready)

Epsilon-Greedy (ε-greedy) is an action selection strategy that balances **exploration and exploitation** by choosing:

- Best action with probability **(1 − ε)**
- Random action with probability **ε**

🔹 Core Idea

👉 "Mostly choose best, sometimes explore"

🔹 Key Concepts

- ε (epsilon): exploration rate
- Greedy action: highest Q-value
- Random action: explore environment

🔹 Formula (Policy)

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A|}, & \text{if } a = a^* \\ \frac{\varepsilon}{|A|}, & \text{otherwise} \end{cases}$$

🔹 Working / Steps

1. Generate random number
2. If < ε → choose random action
3. Else → choose best action
4. Repeat

🔹 Example

ε = 0.1 → 90% best action, 10% random

🔹 Applications

- Game AI
- Robotics
- Recommendation systems
- Online learning
- Ad placement systems

🔶 11. ON-POLICY MC CONTROL

🔹 Definition

On-policy MC control learns the **optimal policy using episodes**, following the **same policy for learning and acting**.

🔹 Core Idea

👉 "Learn policy from its own behavior"

🔹 Key Concepts

- ε-greedy policy
- Episode-based learning
- Policy improvement

🔹 Working / Steps

1. Initialize policy (ε-greedy)
2. Generate episode
3. Compute returns
4. Update Q(s,a)
5. Improve policy
6. Repeat

🔹 Example

Learning strategy in a game using own actions

🔹 Applications

- Game AI
- Simulation systems
- Risk analysis
- Robotics
- Episodic tasks

🔹 Advantages

1. Simple concept
2. No model required
3. Converges to optimal policy
4. Easy to implement
5. Works with exploration

🔹 Disadvantages

1. Needs full episodes
2. Slow learning

3. High variance
4. Not suitable for continuous tasks
5. Requires many samples

🔶 12. ON-POLICY TD LEARNING

🔹 Definition

On-policy TD learning updates values using **current policy actions (e.g., SARSA)**.

🔹 Core Idea

👉 "Learn from actual actions taken"

🔹 Formula (SARSA)

$$Q(s,a) = Q(s,a) + \eta[R + \gamma Q(s',a') - Q(s,a)]$$

🔹 Working

1. Take action A
2. Observe R, S'
3. Choose A'
4. Update Q
5. Repeat

🔹 Applications

- Robotics
- Navigation
- Control systems
- Game AI
- Real-time learning

🔹 Advantages

1. Stable learning
2. Safer policies
3. Works online
4. Handles exploration
5. Good for real-world

🔹 Disadvantages

1. Slower convergence
2. Policy dependent
3. May not reach optimal
4. Needs tuning
5. Sensitive to ε

🔶 13. OFF-POLICY LEARNING

🔹 Definition

Off-policy learning learns optimal policy using **different behavior policy**.

🔹 Core Idea

👉 "Learn from different policy than used"

🔹 Key Concepts

- Target policy (optimal)
- Behavior policy (exploration)

🔹 Example

Q-learning learns optimal policy while exploring randomly

🔹 Applications

- Robotics
- Game AI
- Autonomous systems
- Finance
- Recommendation systems

🔹 Advantages

1. Learns optimal policy
2. More flexible
3. Faster convergence
4. Independent learning
5. Efficient

🔹 Disadvantages

1. Less stable
2. High variance
3. Overestimation problem
4. Complex
5. Needs careful tuning

🔹 Exam Notes

- Off-policy = **different behavior & target policy**

🔶 14. DOUBLE Q-LEARNING

🔹 Definition

Double Q-Learning is an improvement over Q-learning that **reduces overestimation bias** by using **two Q-value tables**.

🔹 **Core Idea**

👉 “Separate action selection & evaluation”

🔹 **Key Concepts**

- Two Q tables: Q1 and Q2
- Alternate updates

🔹 **Update Rule (Concept)**

$$Q_1(s,a) = Q_1(s,a) + \eta[R + \gamma Q_2(s', \arg\max Q_1(s',a)) - Q_1(s,a)]$$

(similar for Q2)

🔹 **Working / Steps**

1. Initialize Q1, Q2
2. Choose action using both
3. Update one table randomly
4. Repeat

🔹 **Example**

Reducing bias in decision making

🔹 **Applications**

- Deep RL
- Game AI
- Robotics
- Finance
- Optimization problems

🔹 **Advantages**

1. Reduces overestimation
2. More accurate
3. Stable learning
4. Better performance
5. Improved convergence

🔹 **Disadvantages**

1. More computation
2. More memory
3. Complex implementation
4. Slower updates
5. Hard to tune

🔷 On-Policy vs Off-Policy (5 Differences)

Feature	On-Policy	Off-Policy
1. Definition	Learns using the **same policy** it follows	Learns using a **different policy** than it follows
2. Learning Source	Uses **actual actions taken**	Uses **best/target actions (optimal)**
3. Policy Type	Behavior policy = Target policy	Behavior policy ≠ Target policy
4. Example	SARSA	Q-Learning
5. Behavior	More **stable and safe**	More **aggressive and optimal**

🔶 15. POLICY GRADIENT METHODS

🔹 Definition (Exam Ready)

Policy Gradient Methods are RL techniques that **directly optimize the policy** by adjusting its parameters using **gradient ascent on expected reward**.

🔹 Core Idea

👉 "Instead of learning value → directly learn policy"
👉 Optimize **πθ(a|s)**

🔹 Key Concepts

- Parameterized policy (θ)
- Gradient ascent
- Expected return maximization
- Stochastic policy

🔹 Objective Function

$$J(\theta) = \mathbb{E}[R]$$

🔹 Policy Gradient Theorem (VERY IMPORTANT)

$$\nabla J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) \cdot Q(s,a)]$$

🔹 Working / Steps

1. Initialize policy parameters θ
2. Sample actions using policy
3. Observe rewards

4. Compute gradient
5. Update θ using gradient ascent
6. Repeat

🔹 **Example**

Learning probability of actions in a game

🔹 **Applications**

- Robotics control
- Autonomous driving
- Game AI (continuous actions)
- Natural language processing
- Recommendation systems

🔹 **Advantages**

1. Works in continuous action space
2. Direct policy optimization
3. Handles stochastic policies
4. Better for complex problems
5. Flexible

🔹 **Disadvantages**

1. High variance
2. Slow convergence
3. Sensitive to learning rate
4. Requires large data
5. Unstable training

🔹 **Exam Notes**

- "Policy Gradient = direct optimization of policy"

🔶 **16. FINITE-DIFFERENCE METHOD**

🔹 **Definition**

Finite-Difference method estimates gradients by **small changes in parameters** and observing change in performance.

🔹 **Core Idea**

👉 "Change θ slightly → observe reward change"

🔹 **Formula**

$$\frac{\partial J}{\partial \theta} \approx \frac{J(\theta + \epsilon) - J(\theta)}{\epsilon}$$

🔹 **Working / Steps**

1. Slightly perturb parameter θ
2. Evaluate reward
3. Compute difference
4. Estimate gradient
5. Update θ

🔹 **Example**

Try slightly different action probability → check reward

🔹 **Applications**

- Optimization problems
- Simulation-based learning
- Robotics tuning
- Control systems
- Black-box optimization

🔹 **Advantages**

1. Simple concept
2. No need for model
3. Works in unknown environments
4. Easy to implement
5. Useful for black-box problems

🔹 **Disadvantages**

1. Very slow
2. Noisy estimates
3. Inefficient
4. High computation cost
5. Not scalable

🔹 **Exam Notes**

- "Finite difference = numerical gradient estimation"

🔶 **17. ACTOR–CRITIC METHODS**

🔹 **Definition**

Actor-Critic is a hybrid RL method where:

- **Actor** → updates policy
- **Critic** → evaluates value function

🔹 **Core Idea**

👉 “Actor decides, Critic evaluates”

🔹 **Key Components**

- Actor (policy πθ)
- Critic (value V or Q)
- TD error

🔹 **Update Rules**

Critic (Value Update)

$$V(s) = V(s) + \eta\delta$$

Actor (Policy Update)

$$\theta = \theta + \eta\nabla_\theta \log \pi_\theta(a \mid s) \cdot \delta$$

🔹 **Working / Steps**

1. Actor selects action
2. Environment gives reward + next state
3. Critic computes TD error
4. Critic updates value
5. Actor updates policy
6. Repeat

🔹 **Example**

Robot learns movement:

- Actor → move
- Critic → evaluate movement

🔹 **Applications**

- Robotics
- Autonomous driving
- Game AI
- Finance
- Deep RL (A3C, PPO)

🔹 **Advantages**

1. Faster learning than pure policy gradient
2. Reduces variance
3. Stable learning
4. Works in continuous spaces

5. Efficient

- **Disadvantages**

1. Complex architecture
2. Requires tuning
3. May be unstable
4. More parameters
5. Hard to implement

- **Exam Notes**

- "Actor = policy, Critic = value"

Value-Based vs Policy Gradient (5 Differences)

Feature	Value-Based Methods	Policy Gradient Methods
1. Learning Target	Learn **value function** (V or Q)	Learn **policy directly** (π)
2. Action Selection	Choose action using **max Q-value**	Choose action using **probability distribution**
3. Policy Type	Mostly **deterministic**	**Stochastic (probabilistic)**
4. Examples	Q-Learning, SARSA	REINFORCE, Actor-Critic
5. Suitability	Works well for **discrete actions**	Works well for **continuous actions**

🔷 UNIT–4: MODEL-BASED LEARNING (REINFORCEMENT LEARNING)

🔶 1. MODEL-BASED REINFORCEMENT LEARNING

🔹 Definition (Exam Ready)

Model-Based Reinforcement Learning is a method where the agent **learns or is given a model of the environment** (transition probabilities and rewards) and uses it for **planning and decision making**.

🔹 Core Idea

👉 "Learn model + plan actions"
👉 Opposite of model-free

🔹 Model Components (VERY IMPORTANT)

- Transition model:

$$P(s' \mid s, a)$$

- Reward model:

$$R(s, a)$$

🔹 Working / Process

1. Agent interacts with environment
2. Learns model (P, R)
3. Uses model to simulate future
4. Applies planning (DP, search)
5. Improves policy

🔹 Example

Robot builds map of environment → plans path

🔹 Applications

- Robotics navigation
- Autonomous driving
- Game AI (planning ahead)
- Industrial automation
- Healthcare decision systems

🔹 Advantages

1. Sample efficient (learns faster)
2. Can plan ahead
3. Better decision making
4. Uses simulations
5. Suitable for structured problems

🔹 Disadvantages

1. Model learning is complex

2. Computationally expensive
3. Model errors affect performance
4. Hard in large environments
5. Requires accurate modeling

🔹 **Exam Notes**

- Model-based = **learn + plan**
- Uses **P(s'|s,a), R(s,a)**

🔶 **2. INTEGRATING PLANNING WITH LEARNING**

🔹 **Definition**

It refers to combining **learning from real experience** with **planning using simulated experience**.

🔹 **Core Idea**

👉 "Learn from real + simulated data"

🔹 **Key Concepts**

- Real experience
- Simulated experience
- Planning updates
- Learning updates

🔹 **Working / Process**

1. Agent collects real data
2. Updates model
3. Simulates experiences
4. Updates value/policy
5. Repeat

🔹 **Example**

Robot learns from real moves + simulated moves

🔹 **Applications**

- Robotics
- Game AI
- Autonomous systems
- Supply chain optimization
- Smart systems

🔹 **Advantages**

1. Faster learning

2. Better data usage
3. Improves performance
4. Efficient updates
5. Combines best of both worlds

🔹 **Disadvantages**

1. Complex system
2. Requires accurate model
3. Computational cost
4. Difficult implementation
5. Error propagation

🔹 **Exam Notes**

- "Real + simulated learning combined"

🔶 **3. INTEGRATED ARCHITECTURE (VERY IMPORTANT)**

🔹 **Definition**

Integrated Architecture is a framework where **learning, planning, and acting are combined together** in one system.

🔹 **Core Idea**

👉 "Agent learns + plans + acts continuously"

🔹 **Components**

1. **Learning module** → updates model
2. **Planning module** → uses model
3. **Acting module** → interacts with environment

🔹 **Working**

1. Observe state
2. Take action
3. Learn from experience
4. Plan using model
5. Update policy

🔹 **Example**

Self-driving car:

- Learns road conditions
- Plans route
- Drives

🔹 **Applications**

- Autonomous vehicles
- Robotics
- AI agents
- Smart cities
- Decision systems

🔹 **Advantages**

1. Continuous improvement
2. Efficient learning
3. Real-time decision making
4. Combines multiple techniques
5. High performance

🔹 **Disadvantages**

1. Complex design
2. High computation
3. Difficult debugging
4. Needs tuning
5. Integration challenges

🔹 **Exam Notes**

- “Learning + Planning + Acting together”

🔶 **4. SIMULATION-BASED SEARCH**

🔹 **Definition**

Simulation-Based Search uses the learned model to **simulate future actions and outcomes** to find the best decision.

🔹 **Core Idea**

👉 “Try in simulation before real action”

🔹 **Key Concepts**

- Tree search
- Simulation
- Rollouts
- Evaluation

🔹 **Working / Steps**

1. Start from current state
2. Simulate possible actions
3. Evaluate outcomes

4. Select best action

🔹 **Example**

Chess AI tries moves before playing

🔹 **Applications**

- Game AI (AlphaGo)
- Robotics
- Path planning
- Finance
- Strategy optimization

🔹 **Advantages**

1. Better decision making
2. Reduces real-world risk
3. Uses future prediction
4. Improves accuracy
5. Powerful for complex problems

🔹 **Disadvantages**

1. Computationally expensive
2. Depends on model accuracy
3. Slow for large problems
4. Requires simulations
5. Complex implementation

🔶 **5. MULTI-ARMED BANDITS (MAB)**

Multi-Armed Bandit Problem in Reinforcement Learning (with real-life example)

The Multi-Armed Bandit (MAB) problem is one of the simplest and most important problems in Reinforcement Learning (RL).

The name comes from a row of slot machines ("one-armed bandits"). Each machine (arm) gives random rewards, and the player must decide which arm to pull to maximize total reward over time.

The main challenge is:

Exploration vs Exploitation

- **Exploration → Try different options to learn their rewards**
- **Exploitation → Choose the option that currently seems best**

Key Components

- **Agent → the decision maker (player, system, student, website, etc.)**
- **Arms → available actions/options**

- **Reward → feedback received after choosing an arm**
- **Goal → maximize cumulative reward**

🔹 **Definition (Exam Ready)**

Multi-Armed Bandit is a simplified RL problem where an agent must **choose among multiple actions (arms)** to maximize **total reward**, without considering state transitions.

🔹 **Core Idea**

👉 "Choose best option among many with uncertainty"
👉 Trade-off: **exploration vs exploitation**

🔹 **Key Concepts**

- Arms (actions)
- Reward distribution
- Expected reward
- Regret minimization

🔹 **Objective**

Maximize cumulative reward:

$$\max \sum_{t} R_t$$

🔹 **Working / Process**

1. Select an arm
2. Observe reward
3. Update estimate
4. Repeat

🔹 **Common Strategies**

- ε-greedy
- Upper Confidence Bound (UCB)
- Thompson Sampling

🔹 **Example**

Choosing best advertisement to show

🔹 **Applications**

- Online advertising
- Recommendation systems
- Clinical trials
- A/B testing

- Website optimization

🔹 **Advantages**

1. Simple framework
2. Efficient learning
3. Fast decision making
4. Useful in real-time systems
5. Strong theoretical base

🔹 **Disadvantages**

1. No state representation
2. Limited to simple problems
3. Cannot model sequences
4. Exploration cost
5. Not suitable for complex RL

🔹 **Exam Notes**

- "Bandit = no states, only actions"

🔶 **6. CONTEXTUAL BANDITS**

🔹 **Definition**

Contextual Bandits extend MAB by including **context (state-like information)** before choosing an action.

🔹 **Core Idea**

👉 "Use context to make better decisions"

🔹 **Key Concepts**

- Context (features)
- Action selection
- Reward prediction

🔹 **Working / Steps**

1. Observe context
2. Choose action
3. Receive reward
4. Update model

🔹 **Example**

Showing ads based on user profile

🔹 **Applications**

- Personalized recommendations

- News feeds
- Healthcare treatment selection
- Marketing
- Search engines

🔹 **Advantages**

1. More informative than bandits
2. Better decisions
3. Real-world applicability
4. Efficient learning
5. Scalable

🔹 **Disadvantages**

1. Still no long-term planning
2. Requires feature engineering
3. Limited sequential learning
4. Model complexity
5. Needs more data

🔹 **Exam Notes**

- Contextual Bandit = **Bandit + context**

🔶 **7. MDP EXTENSIONS**

🔹 **Definition**

MDP Extensions are advanced forms of MDP designed to handle **more complex environments**.

🔹 **Types (Important)**

- POMDP (partial observability)
- Continuous MDP
- Multi-agent MDP
- Hierarchical MDP

🔹 **Core Idea**

👉 "Extend MDP for real-world complexity"

🔹 **Key Concepts**

- Hidden states
- Continuous actions
- Multiple agents
- Hierarchical structure

🔹 **Example**

Autonomous driving with partial information

🔹 **Applications**

- Robotics
- Multi-agent systems
- Smart grids
- Traffic systems
- Finance

🔹 **Advantages**

1. Handles complex environments
2. More realistic modeling
3. Flexible
4. Powerful framework
5. Wide applicability

🔹 **Disadvantages**

1. Very complex
2. High computation cost
3. Difficult to implement
4. Requires approximations
5. Hard to scale

🔹 **Exam Notes**

- “MDP extensions = real-world RL models”

🔶 **8. HIERARCHICAL REINFORCEMENT LEARNING (HRL)**

🔹 **Definition**

Hierarchical RL decomposes a problem into **subtasks (hierarchy)** to simplify learning.

🔹 **Core Idea**

👉 “Break big problem into smaller tasks”

🔹 **Key Concepts**

- High-level policy
- Low-level actions
- Subgoals
- Temporal abstraction

🔹 **Working / Steps**

1. Divide task into subtasks
2. Solve each subtask
3. Combine solutions
4. Learn overall policy

🔹 **Example**

Robot:

- Task → go to kitchen
- Subtasks → move, open door

🔹 **Applications**

- Robotics
- Game AI
- Autonomous systems
- Navigation
- Complex planning

🔹 **Advantages**

1. Reduces complexity
2. Faster learning
3. Reusable skills
4. Scalable
5. Efficient

🔹 **Disadvantages**

1. Designing hierarchy is hard
2. Complex implementation
3. Suboptimal decomposition
4. Requires domain knowledge
5. Coordination issues

🔹 **Exam Notes**

- HRL = **task decomposition**

🔶 **9. SEMI-MARKOV DECISION PROCESS (SMDP)**

🔹 **Definition**

SMDP is an extension of MDP where **actions can take variable time durations**.

🔹 **Core Idea**

👉 "Actions can last multiple time steps"

- **Key Concepts**
  - Temporal abstraction
  - Options (extended actions)
  - Variable time steps
- **Bellman Equation (SMDP)**

$$V(s) = \max_a \mathbb{E}[R + \gamma^\tau V(s')]$$

- **Working**
  1. Choose action (option)
  2. Execute for duration τ
  3. Receive reward
  4. Update value
- **Example**

Driving:

  - Action = “drive to city” (not single step)
- **Applications**
  - Robotics
  - Autonomous driving
  - Task scheduling
  - Industrial automation
  - Game AI
- **Advantages**
  1. Handles long actions
  2. Efficient learning
  3. Better abstraction
  4. Real-world applicability
  5. Reduces decision frequency
- **Disadvantages**
  1. Complex modeling
  2. Hard to define options
  3. Computational cost
  4. Implementation difficulty
  5. Requires tuning
- **Exam Notes**
  - SMDP = **MDP + time duration**

ASSIGNMENT_02

🔷 SECTION–A (1 Mark Each) – Q & A

1. What are the challenges faced when building large-scale reinforcement learning systems?

Answer:
Large-scale RL systems face challenges such as **high computational cost, large state and action spaces, data inefficiency, difficulty in exploration, and training instability**, which make learning slow and complex.

2. What are bootstrapping methods?

Answer:
Bootstrapping methods are techniques in RL where the **value of a state is updated using estimated values of other states**, instead of waiting for final outcomes (e.g., Temporal Difference learning).

3. What is Thompson Sampling?

Answer:
Thompson Sampling is a probabilistic approach used in bandit problems where actions are selected by **sampling from probability distributions of rewards**, balancing exploration and exploitation.

4. What is Return?

Answer:
Return (G) is the **total cumulative discounted reward** received from a given time step onward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots$$

5. What is Monte Carlo Prediction?

Answer:
Monte Carlo Prediction is a method used to estimate value functions by **averaging returns obtained from complete episodes** without using bootstrapping.

6. What do alpha (α) and gamma (γ) represent in Q-Learning?

Answer:

- α (alpha): Learning rate, controls how much new information updates old values
- γ (gamma): Discount factor, determines importance of future rewards

7. What types of Neural Networks do Deep Reinforcement Learning use?

Answer:
Deep RL commonly uses **Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN)** depending on the problem type.

8. Differentiate Episodic tasks vs Continuous tasks?

Answer:

- Episodic tasks: Have a **clear end** (e.g., games)
- Continuous tasks: **Run indefinitely** without terminal state (e.g., robot control)

9. What do you mean by Deep Q-Network (DQN)?

Answer:
Deep Q-Network is an RL algorithm that uses a **deep neural network to approximate Q-values**, enabling learning in large or complex state spaces.

10. What is Bagging and Boosting?

Answer:

- Bagging: Combines multiple models trained **independently** to reduce variance
- Boosting: Combines models trained **sequentially**, focusing on errors to improve accuracy

11. Why was Machine Learning introduced?

Answer:
Machine Learning was introduced to enable systems to **learn patterns from data and improve performance automatically without explicit programming**.

🔷 **SECTION–B (2 MARKS) – Q & A**

1. What is the difference between Learning Rate Decay and Epsilon Decay?

Answer:

- **Learning Rate Decay (α decay):** Gradually reduces the learning rate to make updates more stable over time.
- **Epsilon Decay (ε decay):** Gradually reduces exploration so the agent shifts from exploration to exploitation.
  👉 α controls **learning**, ε controls **action selection**.

2. What is the difference between Deep Q-Network and Categorical Deep Q-Network?

Answer:

- **DQN:** Estimates a single expected Q-value for each action.
- **Categorical DQN:** Estimates a **distribution of possible returns** instead of a single value.
  👉 Categorical DQN provides **more detailed and stable predictions**.

3. How to define states in Reinforcement Learning?

Answer:
States are defined as a representation of the environment that contains **all necessary information for decision-making**.
👉 A good state should satisfy the **Markov property** (no need for past history).

4. What is Experience Replay and what are its benefits?

Answer:
Experience Replay stores past experiences $(s, a, r, s')$and reuses them during training.

Benefits:

- Breaks correlation between data
- Improves learning efficiency
- Stabilizes training
- Reuses past data

5. Can SARSA be used in a POMDP? If yes (or not) why?

Answer:
Yes, SARSA can be used in a POMDP if the agent uses a **belief state or memory (history)** to approximate

the hidden state.

👉 However, performance may be limited due to **partial observability**.

6. What do you understand by Bellman equation in Reinforcement Learning?

Answer:
The Bellman equation defines the value of a state as **immediate reward plus discounted future rewards**, forming a recursive relationship:

👉 "Value = reward + future value"

7. Discuss the limitations of using Value Iteration methods?

Answer:

- Requires full model knowledge (P, R)
- Computationally expensive
- Not scalable to large state spaces
- High memory usage
- Slow convergence for complex problems

8. What is Reward Shaping?

Answer:
Reward Shaping is a technique where **additional intermediate rewards** are provided to guide the agent toward desired behavior and speed up learning.

👉 It helps improve convergence but must be designed carefully to avoid misleading the agent.

9. Can the Monte Carlo method be applicable to all tasks?

Answer:
No, Monte Carlo methods are mainly suitable for **episodic tasks** because they require complete episodes to compute returns.

👉 They are not effective for **continuous or non-terminating tasks**.

10. Give example of Epsilon-Greedy policy in real life?

Answer:
Choosing a restaurant:

- Most of the time you go to your **favorite restaurant (exploitation)**
- Occasionally you try a **new restaurant (exploration)**

👉 This is an example of ε-greedy strategy.

🔷 **SECTION–C (4 MARKS) – Q & A**

1. How does the Monte Carlo prediction method compute the Value Function?

Answer:
Monte Carlo (MC) prediction estimates the value function by **averaging returns from complete episodes**.

Steps:

1. Generate an episode using a policy
2. For each state s, compute return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots$$

3. Average returns over multiple visits:

$$V(s) = \text{Average}(G)$$

Key Points:

- Works only after episode ends
- No bootstrapping
- Uses real experience

2. Advantages of Temporal Difference (TD) over Monte Carlo (MC)?

Answer:

1. **Faster learning** (updates at every step)
2. **Works in continuous tasks** (no need for episode end)
3. **Less memory required**
4. **Online learning possible**
5. **Lower variance** than MC

👉 TD uses **bootstrapping**, while MC waits for full episode.

3. Differentiate between Q-Learning and Policy Gradient methods?

Feature	**Q-Learning**	**Policy Gradient**
Type	Value-based	Policy-based
Learning	Learns Q-values	Learns policy directly
Action	Deterministic (max Q)	Stochastic (probabilities)
Space	Discrete actions	Continuous actions
Example	Q-learning	REINFORCE, Actor-Critic

👉 Q-learning → value first, policy later
👉 Policy Gradient → direct policy optimization

4. Differentiate between Deterministic vs Stochastic Policy?

Feature	**Deterministic Policy**	**Stochastic Policy**
Action	One fixed action	Probability distribution
Output	$a = \pi(s)$	(π(a
Behavior	Predictable	Randomized
Use	Simple problems	Complex/uncertain tasks
Example	Greedy policy	Softmax policy

👉 Deterministic = fixed action
👉 Stochastic = probabilistic action

5. Why choose Policy-Based methods over Value-Based methods?

Answer:

1. Works well in **continuous action spaces**
2. Can learn **stochastic policies**
3. Avoids **value function approximation errors**
4. Better for **complex environments**
5. Provides **direct optimization of policy**

👉 Value-based methods struggle with continuous actions, while policy methods handle them easily.

🔷 **SECTION–D (6 MARKS) – Q & A**

1. Describe Deep Q-Networks (DQN) and Experience Replay in Reinforcement Learning

🔹 **Deep Q-Network (DQN) – Definition**

DQN is a Reinforcement Learning algorithm that uses a **deep neural network to approximate the Q-value function**, enabling learning in large and complex state spaces.

🔹 **Key Idea**

👉 Combines **Q-learning + Deep Learning**

🔹 **Working / Steps**

1. Input state into neural network
2. Network outputs Q-values for all actions
3. Select action using ε-greedy
4. Execute action → get reward and next state
5. Update network using loss function

🔹 **Q-Learning Update Rule**

$$Q(s,a) = Q(s,a) + \alpha[R + \gamma \max\ Q(s',a') - Q(s,a)]$$

🔹 **Experience Replay – Definition**

Experience Replay stores past experiences (s, a, r, s') in a **memory buffer** and reuses them during training.

🔹 **Working**

1. Store experience in memory
2. Sample random mini-batch
3. Train network using sampled data

🔹 **Benefits of Experience Replay**

- Breaks correlation between samples

- Improves data efficiency
- Stabilizes learning
- Reuses past experiences
- Enables batch learning

🔹 **Applications**

- Game AI (Atari, Chess)
- Robotics
- Autonomous driving
- Recommendation systems

2. Define Exploration and Exploitation in Reinforcement Learning

🔹 **Definition**

- **Exploration:** Trying new actions to discover better rewards
- **Exploitation:** Choosing the best-known action to maximize reward

🔹 **Core Idea**

👉 Balance between **learning new things** and **using known knowledge**

🔹 **Example**

- Trying a new restaurant → Exploration
- Going to favorite restaurant → Exploitation

🔹 **Methods to Handle Trade-off**

- ε-greedy strategy
- Softmax action selection
- Upper Confidence Bound (UCB)

🔹 **Importance**

- Prevents getting stuck in local optimum
- Improves long-term performance

3. Explain Applications of Hierarchical Reinforcement Learning (HRL) in detail

🔹 **Definition**

Hierarchical Reinforcement Learning breaks a complex task into **smaller sub-tasks (hierarchy)** to simplify learning.

🔹 **Core Idea**

👉 "Divide and solve complex problems"

🔹 **Working**

1. Decompose task into subtasks
2. Learn each subtask

3. Combine results
4. Form overall policy

🔹 **Applications (Detailed)**

1. Robotics

- Robot performs tasks like navigation, object picking
- Subtasks: move → detect → grab

2. Autonomous Driving

- Tasks divided into:
    - Lane detection
    - Speed control
    - Obstacle avoidance

3. Game AI

- Strategy planning → attack → defense
- Improves performance in complex games

4. Natural Language Processing

- Sentence generation → word selection → grammar control

5. Industrial Automation

- Task scheduling → execution → monitoring

🔹 **Advantages**

1. Reduces complexity
2. Faster learning
3. Reusable skills
4. Scalable
5. Efficient decision making

🔹 **Disadvantages**

1. Difficult to design hierarchy
2. Requires domain knowledge
3. Complex implementation
4. Coordination issues
5. Suboptimal decomposition possible